words in texts; if that's the case, we often need to know either how many occurrences of a word appear in the text, or the positions where they appear: in the former situation we can store a counter in each node, and only key nodes will have a positive value; in the latter we will instead use a list of positions, storing the index, in the text, where each occurrence starts.

6.2.1 Why is it Better Again?

Let's address space first: why the trie in figure 6.3 is better than the binary search tree in figure 6.1?

Let's do some quick math… but first, we need to make some assumptions:

- We only consider ASCII strings and characters, so we have to account for 1 byte for each char (UNICODE wouldn't change much, and rather make the cheaper option even cheaper, in comparison to the other) plus, in BSTs, 1 byte for each string's terminator.
- We only explicitly store links to actual nodes in tries, and account for a fixed number of bytes for null-pointers in BSTs (same space taken by non-null references).
- As we mentioned, each node in a trie has $|\sum|$ links, where $|\sum|$ is the alphabet size: this means that in tries, and especially in nodes in the lower levels, most links are `null`, and indeed in listing 6.1 it's also possible to see how all those links are initialized in `Node`'s constructor.
- For each node in a trie, we account for a fixed amount of space to store the children list (we can imagine that we